Argo CD - GitOps từ commit đến một lần đồng bộ có kiểm chứng

Một chủ đề, một tài liệu: mô hình, bài tập, lỗi thường gặp và câu hỏi phỏng vấn.

Đọc theo thứ tự trong tập. Làm lab trên môi trường riêng, ghi kết quả và đối chiếu tài liệu chính thức theo phiên bản bạn đang dùng.

Bám theo bốn JD người học cung cấp | 11/10/2026

Tập này học sau Git, CI/CD cơ bản, Kubernetes Deployment/Service và Helm ở mức đọc manifest. Mục tiêu là bạn tự giải thích được Argo CD lấy trạng thái mong muốn ở đâu, so với cluster thế nào, khi nào đồng bộ, cách phát hiện drift, cách xử lý lỗi và cách rollback an toàn. Tài liệu dùng cluster local; ví dụ không đòi cloud. **Đọc PDF xong chưa được tính là biết vận hành Argo CD:** phải làm lab, đọc status/diff và giải thích một lỗi do bạn tự tạo.

1. Bài toán phát hành mà GitOps giải quyết

Một pipeline có thể build image và chạy test tốt, nhưng câu hỏi tiếp theo là: ai quyết định image nào chạy ở staging/production, manifest nằm đâu, ai được sửa, và nếu có người **kubectl edit** trực tiếp thì bản nào là đúng? GitOps chọn Git làm nơi lưu **desired state** của môi trường. Mỗi thay đổi đi qua commit/review; một controller trong cluster đọc Git, so với live state và đưa cluster về trạng thái mong muốn theo chính sách.

Ví dụ Notes API: CI build image từ commit **a1b2c3**, đẩy lên registry và lưu digest. Một PR ở repository triển khai đổi digest trong manifest/Helm values. Sau review và merge, Argo CD đọc revision mới, render manifest, so với Deployment đang chạy và sync. Sau sync, smoke test gọi **POST /notes**, telemetry theo dõi lỗi/độ trễ. Khi có sự cố, ta biết commit mã nguồn, digest image, commit cấu hình và thời điểm sync.

```
Repo mã nguồn -> CI/test/build -> image digest -> registry
                                        |
                                PR repo triển khai
                                        |
                                review/merge vào Git
                                        |
                        Argo CD compare -> sync -> Kubernetes
                                        |
                          smoke test + metrics/logs + rollback
```

CI chịu trách nhiệm kiểm tra và tạo artifact; Argo CD chịu trách nhiệm đồng bộ desired manifest. Helm có thể render chart thành manifest trước khi Argo áp. Tách trách nhiệm giúp quyền pipeline không cần trực tiếp **kubectl apply** vào production. Tuy nhiên quyền merge vào repo triển khai trở nên rất mạnh; branch protection, review và AppProject policy là phần của bảo mật hệ thống.

2. Mô hình vòng lặp: desired, live, diff và sync

Desired state là resource YAML sau khi Argo render từ source (plain YAML, Helm, Kustomize...). **Live state** là resource Kubernetes hiện có. Argo so hai trạng thái và hiển thị khác biệt. Nếu khác, Application có thể **OutOfSync**. Khi sync, Argo áp thay đổi qua Kubernetes API theo strategy/options. Việc so sánh và sync không đồng nghĩa ứng dụng phục vụ người dùng đúng; nó chỉ giải quyết lớp trạng thái resource.

Lấy ví dụ Git khai báo Deployment replicas=2; trong cluster có replicas=1. Argo hiển thị diff ở trường replicas. Nếu sync thủ công, người vận hành xem diff rồi cho áp. Nếu auto-sync và điều kiện phù hợp, Argo tự áp. Nếu có HPA cũng liên tục thay replicas, hai controller có thể tranh quyền sở hữu trường. Cần quyết định HPA hay Git quản lý replicas và cấu hình manifest/ignore differences cho phù hợp, không chỉ bật self-heal rồi để hai vòng lặp kéo qua kéo lại.

Một tình huống khác: có người sửa image bằng **kubectl set image**. Live khác Git. Nếu self-heal bật, Argo có thể kéo lại image trong Git. Nếu đó là can thiệp khẩn cấp, phải ghi incident và đưa thay đổi vào Git hoặc có quy trình tạm dừng đồng bộ; nếu không, lần reconcile có thể xóa tác dụng của sửa tay. Drift là tín hiệu về ownership/quy trình, không chỉ màu đỏ trên dashboard.

3. Application: cấu hình mà Argo đọc

Application là resource mô tả project, source và destination. Đây là mẫu lab dựa trên repository ví dụ của dự án Argo CD:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: guestbook-lab
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/argoproj/argocd-example-apps.git
    targetRevision: HEAD
    path: guestbook
  destination:
    server: https://kubernetes.default.svc
    namespace: guestbook-lab
  syncPolicy:
    syncOptions:
      - CreateNamespace=true
```

Đọc lần lượt: **metadata.namespace=argocd** là nơi **Application object** nằm, không phải nơi app chạy. **project** xác định ràng buộc source/destination/resource. **repoURL** là nguồn Git; **targetRevision** là ref; **path** chọn thư mục manifest. **destination.server** chỉ cluster; **destination.namespace** là nơi tạo Deployment/Service. **CreateNamespace=true** cho phép tạo namespace đích khi sync nếu Argo có quyền. Mẫu chưa bật **automated**, vì thế ta quan sát diff và sync chủ động trong lab.

HEAD tiện cho ví dụ public nhưng không phải chiến lược production tốt nếu bạn cần tái hiện chính xác release. Hãy dùng branch được bảo vệ hoặc commit/ref rõ ràng và ghi revision đã sync. Mẫu cũng dùng project **default**; tài liệu Argo CD cho biết project mặc định ban đầu khá rộng. Môi trường nhiều nhóm nên tạo AppProject có danh sách repository, destination và resource được phép, kèm quyền người dùng phù hợp.

Bài tập đọc YAML trước khi chạy

Không nhìn lại mẫu, trả lời: nếu đổi **metadata.namespace** thành **guestbook-lab**, Argo CD mặc định có nhìn thấy Application không? Nếu đổi **destination.namespace** thành **production**, rủi ro gì? Nếu path sai, Pod có được tạo không? Bạn cần xác nhận cấu hình Argo CD đang dùng vì việc đặt Application ngoài namespace **argocd** có thể được cấu hình riêng. Vấn đề quan trọng là phân biệt namespace của **bộ điều khiển** và namespace của **workload**.

4. Kiến trúc bên trong Argo CD

Repo server lấy source và tạo manifest từ Git/Helm/Kustomize. Nếu path sai, Git credential hết hạn hoặc render lỗi, bước so sánh có thể thất bại trước khi có bất cứ thay đổi nào ở cluster. **Application controller** theo dõi Application, so desired với live, quản lý sync/status. **API server/UI/CLI** cho người vận hành xem và thao tác theo quyền. Argo CD dùng Kubernetes API để đọc/ghi tài nguyên đích; nếu RBAC hoặc AppProject không cho, sync có thể thất bại dù Git hoàn toàn đúng.

Đường điều tra một lỗi vì thế gồm: Git có đúng revision/path không -> repo server render được không -> diff ra sao -> AppProject/RBAC cho phép không -> Kubernetes API/admission chấp nhận không -> Pod/Service có khỏe không -> request người dùng có thành công không. Đi theo thứ tự này giúp tránh sửa app khi manifest còn chưa được render, hoặc sửa Git khi thực ra registry không cho kéo image.

5. Hai trục trạng thái: Sync và Health

Sync status so desired với live: **Synced**, **OutOfSync** hoặc trạng thái khác khi chưa xác định được. **Health status** đánh giá trạng thái của resource theo bộ quy tắc Argo: **Healthy**, **Progressing**, **Degraded**... Một Application có thể **Synced** nhưng **Degraded**: manifest đúng Git, Deployment lại thiếu replica Ready. Có thể **OutOfSync** nhưng app vẫn phục vụ request: ai đó scale trực tiếp hoặc HPA thay trường được theo dõi.

Synced/**Healthy** vẫn chưa chứng minh chức năng nghiệp vụ. Một Deployment có 2 Pod Ready và Service có endpoint, nhưng **POST /notes** trả 500 vì database hết chỗ; probe **/health** quá nông. Cần smoke test qua đường người dùng, metrics, logs và nếu có thì trace. Đừng dùng hai màu xanh của Argo làm cổng phát hành duy nhất.

Ma trận đọc trạng thái

Sync	Health	Cách hiểu ban đầu	Bước kiểm tra
Synced	Healthy	Resource giống Git và qua health rules	Smoke test nghiệp vụ, telemetry
Synced	Degraded	Manifest đúng nhưng workload hỏng	Pod Events/log, image, probe, resource
OutOfSync	Healthy	Cluster khác Git nhưng app có thể chạy	Xem diff, xác định owner thay đổi
Unknown/ComparisonError	Unknown	Chưa render/so được	Git/ref/path, credential, repo server

Đây là cách đọc ban đầu, không phải bảng kết luận. Status có thể chậm hơn sự kiện thực tế trong một khoảng ngắn; xem thời điểm refresh/sync và Kubernetes resource trực tiếp khi điều tra sự cố.

6. Từ thay đổi Git đến deployment: từng bước có thể kiểm chứng

Giả sử phiên bản đang chạy là A. Developer tạo pull request đổi image thành B. CI chạy test, build image B và đẩy lên registry; một bước có kiểm soát cập nhật image digest trong repository cấu hình. Người duyệt xem diff, bằng chứng kiểm thử và chấp thuận merge. Sau merge, Argo CD nhận biết revision mới, render manifest, so với cluster và đánh dấu **OutOfSync**. Với manual sync, người có quyền xem diff rồi ra lệnh sync. Với auto sync, controller áp dụng thay đổi theo chính sách đã cấu hình. Kubernetes tiếp tục rollout; Argo quan sát health. Cuối cùng đội sản phẩm kiểm tra smoke test và tín hiệu vận hành. Nếu bỏ qua bước CI hoặc cập nhật Git trước khi image tồn tại, Pod có thể **ImagePullBackOff** dù sync thành công về mặt cấu hình.

Hãy viết lại chuỗi này bằng hai cột **desired state** và **live state** tại các thời điểm: trước merge, sau merge trước sync, sau sync trước rollout, sau rollout. Đây là cách hiểu **OutOfSync** mà không học thuộc màu của giao diện.

Sync không phải deploy thành công

Một sync có thể **Succeeded** vì Kubernetes API chấp nhận Deployment, nhưng rollout chưa hoàn tất. Argo có health assessment riêng; health cũng có thể xanh trước khi kiểm thử nghiệp vụ đầy đủ. Vì thế release gate cần có ít nhất: revision/digest rõ ràng, sync result, rollout status, smoke test theo đường người dùng, metrics lỗi/độ trễ trong khoảng quan sát và cách quay lui. Với thay đổi database, kế hoạch tương thích schema còn quan trọng hơn nút rollback.

Prune, self-heal, auto-sync

prune cho phép Argo xóa tài nguyên live không còn trong source sau sync. Ví dụ xóa Service khỏi Git có thể khiến traffic mất ngay khi prune. **selfHeal** khiến thay đổi trực tiếp trong cluster bị đưa trở lại trạng thái Git theo chính sách tự đồng bộ. Auto-sync giúp giảm thao tác tay nhưng làm lỗi Git lan đến cluster nhanh hơn. Ở lab đầu tiên, hãy để manual sync và chưa bật prune. Sau khi hiểu diff, policy và cách khôi phục, mới thử auto-sync trong namespace thử nghiệm. Các tùy chọn và hành vi retry phụ thuộc phiên bản, nên kiểm tra tài liệu automated sync trước khi áp dụng cho production.

Một lệnh **kubectl edit** khi chữa cháy tạo **drift**. Ghi lại lý do, thời điểm, người thực hiện và cách đưa sửa đổi vào Git. Nếu chỉ sửa live object, lần sync tiếp theo có thể xóa bản vá. Nếu tắt tự đồng bộ mà không ghi nhận, Git và thực tế tách xa nhau; đó là nợ vận hành.

7. Ranh giới quyền: AppProject, tài khoản và secrets

Argo CD có quyền thay đổi tài nguyên Kubernetes. Vì vậy một pull request tới repository cấu hình có thể tương đương quyền triển khai. Bảo vệ branch, review và CI không phải phần trang trí. AppProject giới hạn source repository, destination cluster/namespace và nhóm tài nguyên có thể dùng; RBAC của Argo giới hạn ai được đọc/sync/sửa Application. Kubernetes RBAC và admission policy vẫn là lớp kiểm soát ở cluster. Đội nhiều tenant phải thiết kế cả ba lớp, không chỉ tạo nhiều namespace.

Hãy nghĩ về ba câu hỏi khi review một Application: (1) Nó lấy mã từ đâu và ai có thể sửa mã đó? (2) Nó được phép ghi vào cluster/namespace nào? (3) Nó có thể tạo loại resource gì, ví dụ RoleBinding, Ingress hay CRD? **default** AppProject ban đầu khá rộng theo tài liệu Projects; không dùng mặc định làm bằng chứng phân quyền đã đủ.

Git không nên chứa token, mật khẩu hay kubeconfig dạng rõ. Một Kubernetes Secret được mã hóa base64 trong YAML **không phải** bảo vệ bí mật. Dùng một hệ thống quản lý secrets phù hợp và cấp quyền đọc tối thiểu cho công cụ đồng bộ. Trong lab công khai bên dưới không có secret. Nếu sau này dùng Vault/External Secrets, phân biệt rõ secret trong Git, secret được tham chiếu trong Git, và quyền của controller lấy secret thật.

Phê duyệt là một phần kiến trúc

Một pipeline đủ trưởng thành phải trả lời: ai có thể merge manifest production, ai được sync, ai được bỏ qua gate, và hành động khẩn cấp được ghi lại ở đâu? Chia quyền giữa người viết code, người review và người deploy ở môi trường nhạy cảm; dùng audit log và quy trình break-glass có thời hạn. Cụ thể hóa trong repository bằng branch protection và policy, không chỉ ghi trong wiki.

8. Lab: cài Argo CD trên cụm local và triển khai một ứng dụng

Mục tiêu lab là nhìn thấy cùng một ứng dụng qua ba nơi: Git, Argo CD và Kubernetes. Bạn sẽ tạo namespace đích, khai báo Application, xem diff, sync thủ công, kiểm tra request, rồi tạo drift và khôi phục. Không cần tài khoản cloud. Máy cần Docker, kind, kubectl, Git và mạng để kéo image/manifest public. Dùng cụm lab trống; kiểm tra context để tránh áp dụng nhầm vào cluster công ty.

Bước 0: kiểm tra môi trường

```
kind version
kubectl version --client
kubectl config current-context
kubectl get nodes
```

Nếu chưa có cluster lab, **kind create cluster --name devops-study** rồi kiểm tra context là **kind-devops-study**. **kubectl get nodes** phải thấy control-plane **Ready** trước khi tiếp tục. Nếu bạn đang dùng Linux/macOS, lệnh kubectl/kind giống nhau; cú pháp shell khác chỉ ở biến môi trường và quản lý tiến trình. Không tiếp tục nếu context là production hoặc một cluster không thuộc lab.

Bước 1: cài Argo CD

Theo Getting Started chính thức, tạo namespace và áp manifest cài đặt:

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests
    /install.yaml
kubectl get pods -n argocd
kubectl rollout status deployment/argocd-server -n argocd --timeout=5m
```

URL **stable** thay đổi theo release; cho môi trường thật hãy ghim version, đọc release notes và lưu lại manifest/chart đã duyệt. Trong lab, các Pod có thể cần vài phút kéo image. Nếu chưa **Running/Ready**, xem **kubectl describe pod -n argocd <ten-pod>** và Events trước khi thử lại. Không kết luận cài lỗi chỉ vì kết quả lệnh đầu tiên xuất hiện **ContainerCreating**.

Để mở UI trên máy riêng, chạy **kubectl port-forward svc/argocd-server -n argocd 8080:443** trong terminal khác, rồi truy cập **https://localhost:8080**. Trình duyệt có thể cảnh báo chứng chỉ tự ký trong lab. Làm theo hướng dẫn chính thức về mật khẩu admin ban đầu; không chép mật khẩu vào tài liệu, issue hoặc ảnh chụp. Sau lab hãy xóa cluster hoặc đổi/khóa tài khoản nếu giữ lại.

Bước 2: tạo Application

Tạo file **lesson/labs/argocd-local/guestbook-lab.yaml** như phụ lục đi kèm PDF, rồi chạy:

```
kubectl apply -f lesson/labs/argocd-local/guestbook-lab.yaml
kubectl get application guestbook-lab -n argocd
kubectl describe application guestbook-lab -n argocd
```

Application dùng repository ví dụ chính thức **argoproj/argocd-example-apps**, thư mục **guestbook**, namespace đích **guestbook-lab**, project **default** và manual sync. Repository public có thể thay đổi theo thời gian; trước khi lab, mở URL nguồn trong phụ lục và xác nhận thư mục tồn tại. Nếu cluster không có Internet, repo-server không render được; đó là lỗi nguồn, chưa phải lỗi Deployment.

Đợi Argo refresh, mở Application trong UI và xem **Source**, **Destination**, **Sync status**, **Health** và cây resource. Trước sync, bạn thường thấy **OutOfSync** và resource dự kiến. Nếu báo **ComparisonError**, xem condition chi tiết; không đoán từ biểu tượng. Chọn **Sync**, xem danh sách thay đổi và đồng bộ. Sau đó kiểm tra:

```
kubectl get deployment,service,pods -n guestbook-lab
kubectl rollout status deployment/guestbook-ui -n guestbook-lab --timeout=5m
kubectl get endpointslice -n guestbook-lab
```

Kết quả kỳ vọng là Deployment đạt số replica Ready và Service có EndpointSlice chứa địa chỉ Pod phù hợp. Tên/port cụ thể phải đọc từ manifest của revision đang dùng, vì repository ví dụ có thể được cập nhật. Nếu Deployment không có, kiểm tra Application diff và operation state. Nếu Deployment có nhưng Pod chưa Ready, chuyển sang **kubectl describe pod** và logs; không sửa Argo CD trước khi biết tầng lỗi.

Bước 3: kiểm tra đường người dùng

Đọc Service port bằng **kubectl get svc guestbook-ui -n guestbook-lab -o yaml**, rồi port-forward từ Service đến một cổng local phù hợp, ví dụ **kubectl port-forward svc/guestbook-ui -n guestbook-lab 8081:80** nếu port Service là 80. Mở **http://localhost:8081** và kiểm tra trang. Nếu port khác, thay số sau dấu **:**. Đây là smoke test tối thiểu: nó đi qua Service và Pod; chưa kiểm tra ingress, TLS hay dữ liệu bền vững. Ghi lại revision đang sync và kết quả quan sát thực tế trong ghi chú lab của bạn.

Bước 4: tạo drift có kiểm soát

Xác nhận manifest nguồn đặt replica là 1 rồi chạy **kubectl scale deployment/guestbook-ui -n guestbook-lab --replicas=2**. Đợi Application refresh. Quan sát diff: live có 2, desired có 1, do đó thường **OutOfSync**; ứng dụng vẫn có thể **Healthy**. Nếu **selfHeal** đã bật ngoài ý muốn, cluster có thể tự quay lại 1 trước khi bạn quan sát; kiểm tra spec Application. Trong UI, sync lại và kiểm tra số replica. Bạn vừa chứng minh Git là nguồn desired state và Argo có thể sửa drift. HPA có thể cũng thay replicas; production cần cấu hình tránh hai controller tranh chấp trường này.

Bước 5: gây lỗi render an toàn

Lưu file Application ban đầu. Trong bản sao, đổi **spec.source.path** thành một thư mục không tồn tại rồi **kubectl apply -f <ban-sao>**. Đợi refresh và đọc **ComparisonError**/condition. Pod cũ có thể vẫn chạy vì Argo chưa có manifest mới hợp lệ; đây là một đặc điểm quan trọng, không phải dấu hiệu lỗi được sửa. Áp lại file ban đầu, refresh, xác nhận comparison trở lại bình thường. **Không bật prune** trong bài này. Nếu tài liệu/phiên bản hiện tại hiển thị thông báo khác, ghi nguyên văn thông báo và giải thích theo bước render -> compare -> sync.

Bước 6: dọn lab

Xóa Application và namespace chỉ sau khi kiểm tra context lab: **kubectl config current-context**; **kubectl delete application guestbook-lab -n argocd**; **kubectl delete namespace guestbook-lab**. Nếu tạo riêng cluster bằng kind, **kind delete cluster --name devops-study**. Việc xóa Application có thể chịu ảnh hưởng finalizer/cascade policy; nếu namespace còn resource, xem **kubectl get all -n guestbook-lab** và policy thay vì xóa cưỡng bức. Giữ lại file YAML và ghi chú, không giữ cluster tốn tài nguyên nếu không cần.

9. Chẩn đoán sự cố theo tầng

Triệu chứng	Câu hỏi phân biệt	Nơi xem trước
Application không xuất hiện	Object tạo ở namespace nào? Controller có theo dõi?	**kubectl get applications -A**, UI filter
ComparisonError	Clone Git hay render Helm/Kustomize lỗi?	Application conditions, repo-server logs, URL/ref/path
OutOfSync kéo dài	Manual sync chưa chạy, diff bị ignore hay sync bị từ chối?	Diff và operation history
Sync failed, permission denied	AppProject, Argo RBAC, Kubernetes RBAC hay admission?	Operation message, Project, Kubernetes Events
Synced nhưng **Degraded**	Pod thiếu Ready vì image, probe, tài nguyên hay mount?	**kubectl describe pod**, logs, rollout
Synced và **Healthy** nhưng 5xx	Smoke test, DB, dependency hay route có vấn đề?	Service/Ingress, ứng dụng, logs, metrics, traces

Điều tra bằng **một thay đổi, một giả thuyết, một bằng chứng**. Chép message lỗi chính xác; ghi **targetRevision** và revision thực đã sync. Nếu nhiều Application cùng lỗi clone repo, nghĩ về credential/DNS/network của repo-server. Nếu chỉ một app lỗi, ưu tiên path, ref và manifest của app đó. Nếu nhiều app sync được nhưng Pod không kéo image, nghĩ về registry, node network hoặc credential image pull. Cách nhóm triệu chứng giúp tìm lỗi chung nhanh hơn.

Tình huống phỏng vấn: Git đúng, Argo xanh, khách hàng vẫn lỗi

Đừng trả lời "Argo CD không đáng tin". Hỏi: request đi qua DNS, load balancer, ingress, Service và Pod nào; **/health** kiểm tra gì; thay đổi DB có tương thích không; tỷ lệ 5xx và latency bắt đầu từ revision nào? Xem dashboard theo revision, log/trace của một request lỗi, chạy smoke test theo chức năng thực. Cân nhắc rollback image nếu migration còn tương thích; nếu dữ liệu/schema không thể đảo, dùng roll-forward và kế hoạch phục hồi đã thử. Argo là một mắt xích trong chuỗi bằng chứng.

10. Helm, Kustomize, ApplicationSet và multi-environment

Argo có thể render raw YAML, Kustomize và Helm. Với Helm, Argo render chart thành manifest rồi quản lý trạng thái ứng dụng; cần hiểu ranh giới giữa **chart version**, **values**, **image version/digest** và **Application revision**. Một thay đổi values có thể làm manifest đổi mà code app không đổi. Đừng giấu cấu hình production trong tham số thao tác tay khó review; lưu cấu hình môi trường trong Git với chủ sở hữu rõ.

Kustomize thích hợp cho base/overlay khi chỉ cần biến thể manifest; Helm hữu ích khi cần đóng gói chart và templates. Không cần dùng cả hai cho mọi app. Trước khi chuẩn hóa một công cụ, hỏi: một người mới có thể lần từ Application đến manifest cuối cùng và giải thích vì sao giá trị đó xuất hiện không? Render output trong CI để review dễ hơn.

ApplicationSet tạo nhiều Application từ generator (ví dụ danh sách cluster hoặc thư mục). Nó giảm sao chép nhưng mở rộng phạm vi tác động của một lỗi template. Với multi-cluster, hãy thử ở một môi trường, giới hạn project/destination, quan sát diff trên một Application sinh ra, rồi mới tăng phạm vi. Không coi “một file tạo được trăm app” là thành công nếu không có cơ chế cô lập lỗi và rollback.

11. Ca release có lời giải: một thay đổi image và hai kiểu lỗi

Mục tiêu: học phân biệt lỗi ở bước render, bước áp manifest và bước ứng dụng chạy. Giả sử production hiện chạy image digest A, Git chứa cấu hình A; một PR đổi sang digest B. CI đã build và đẩy B vào registry. Branch protection yêu cầu review, chart/manifest render được trong CI. Người phê duyệt merge, Argo thấy revision mới và **OutOfSync**.

Kịch bản 1: đường dẫn Helm values sai

Trước sync, repo-server báo lỗi không tìm thấy file values. Không có manifest desired mới để áp. Nếu app cũ còn chạy, traffic vẫn có thể bình thường. Người trực ca phải đọc Application conditions và log render, xác định path/revision và sửa ở Git. Bấm Sync lại nhiều lần không làm xuất hiện file. Đổi trực tiếp Deployment live để “chạy B” tạo drift và bỏ qua review; nếu buộc phải can thiệp khẩn cấp, cần quy trình ghi nhận và đồng bộ lại nguồn.

Sau khi sửa path, kiểm tra diff **toàn bộ** manifest đã render. Một file values sai có thể đổi không chỉ image mà cả replicas, env, probes hoặc resource limits. Đọc diff là bước bảo vệ cuối trước khi apply, nhất là khi chart có nhiều templates.

Kịch bản 2: sync thành công, Pod mới không kéo được image

Argo có thể báo sync operation thành công vì Deployment object được API nhận; resource health sau đó **Progressing** hoặc **Degraded**, Pod mới **ImagePullBackOff**. Đây là lỗi ở node/registry/image reference/credential, không phải repo-server. Đọc Events của Pod để phân biệt **not found**, **unauthorized**, timeout, TLS hay DNS. Nếu digest B không tồn tại trong registry, sửa pipeline/phát hành artifact trước. Nếu registry private từ node không tới được, sửa đường mạng/quyền hoặc quay về A để giảm tác động.

Giả sử ReplicaSet cũ vẫn giữ hai Pod A, smoke test có thể còn thành công. Điều này không có nghĩa release B thành công. Ghi image digest thực ở Pod hiện đang phục vụ và số replica A/B. Nếu chuyển

sang B mà readiness quá nông, người dùng có thể gặp lỗi trước khi Argo đánh giá **Degraded**; vì vậy quan sát error rate và chức năng cốt lõi theo phiên bản.

Quyết định rollback có điều kiện

Trong GitOps, cách bền vững là revert commit cấu hình B hoặc tạo commit đưa manifest về digest A, qua review phù hợp; Argo sync revision đó. Với incident cần hành động nhanh, có thể dùng khả năng rollback/sync revision cũ theo runbook, nhưng phải giải quyết tiếp source Git để controller không đưa B trở lại. Nếu release B kèm migration DB không tương thích A, đổi image về A có thể tệ hơn; cần xác nhận kế hoạch expand/contract và dữ liệu trước khi bấm nút. Ghi rõ ai quyết định, bằng chứng nào và kết quả sau 5/15/30 phút quan sát.

Bảng xác định tầng hỏng

Bằng chứng	Tầng đang hỏng	Hành động đầu tiên
ComparisonError, không render được	Source/render	Kiểm tra URL/ref/path/Helm values và repo-server
Operation denied/forbidden	Policy/API	AppProject, RBAC, admission, namespace
Sync succeeded, Pod pull failed	Runtime/registry	Pod Events, digest, quyền registry, node network
Synced/Healthy, business 5xx	Application/dependency	Smoke test, logs, metrics, trace, DB

Một báo lỗi có thể bị che bởi báo lỗi khác. Luôn bắt đầu từ điều kiện sớm nhất trong chuỗi Git -> render -> compare -> apply -> rollout -> request, rồi mới điều tra phía sau.

12. Thiết kế staging và production có thể truy vết

Một mô hình dễ hiểu cho nhóm nhỏ là cùng một chart/manifest base, hai overlay hoặc values file riêng cho staging và production, mỗi môi trường có Application riêng. CI build **một artifact bất biến** có digest; staging dùng digest đó và chạy integration/smoke; PR production chỉ thay digest/values đã review. Tránh rebuild một image mới từ cùng tag khi lên production: bạn sẽ mất bằng chứng hai môi trường dùng cùng binary.

Trong Application, **targetRevision: HEAD** trên branch thay đổi liên tục giúp thử nghiệm nhanh nhưng khó tái hiện nếu không ghi revision đã sync. Production nên có chính sách rõ về branch/tag/commit, review, promotion và rollback. Một số nhóm dùng monorepo môi trường, nhóm khác dùng repository tách riêng; điểm bắt buộc là có thể trả lời “ai duyệt thay đổi X, manifest render ra sao, artifact digest nào đang chạy, cluster nào đã sync, và người dùng thấy gì”.

Không tạo hàng chục Application copy-paste rồi hy vọng chúng giống nhau. Template hoặc ApplicationSet có thể sinh chúng, nhưng phải kiểm tra phạm vi của generator và cho người review xem manifest cuối. Nếu một template đổi **destination.namespace** sai, tác động có thể lan mọi môi trường. Tách quyền/approval và dùng AppProject để một Application staging không thể ghi production, kể cả khi một PR bị sai.

Khi nào cần auto-sync?

Ở dev/staging, auto-sync giúp phản hồi nhanh; ở production, quyết định phụ thuộc rủi ro, chất lượng CI, canary, quyền review và khả năng rollback. Manual sync không tự động an toàn nếu người bấm không đọc diff. Auto-sync không tự động nguy hiểm nếu guardrail và quan sát tốt. Viết điều kiện cụ thể: branch được bảo vệ, CI bắt buộc, image đã tồn tại, render/diff được kiểm tra, chính sách prune rõ, SLO/alert và người chịu trách nhiệm. Khi một điều kiện chưa có, thu hẹp phạm vi tự động hóa trước.

13. Câu hỏi tự kiểm tra có lời giải

Câu 1. Git mong 1 replica, **kubectl scale** lên 2, app vẫn chạy. Argo báo gì? **Lời giải:** nếu trường replicas được so sánh và không bị ignore, Application có thể **OutOfSync** trong khi Health vẫn **Healthy**. Nếu HPA là chủ quản replicas, cần thiết kế diff/ownership khác; không kết luận chỉ từ ví dụ.

Câu 2. Application **Synced** và **Healthy**; người dùng vẫn nhận HTTP 500. Có nên restart repo-server? **Lời giải:** chưa có bằng chứng repo-server lỗi. Xem đường request và log/metric/trace của app, DB/queue và revision vừa phát hành. Hai trạng thái Argo mô tả resource, không kiểm thử nghiệp vụ.

Câu 3. Xóa Deployment YAML khỏi Git và bật **prune**; điều gì có thể xảy ra? **Lời giải:** trong sync, Argo có thể xóa Deployment live. Nếu Service vẫn tồn tại, nó có thể hết backend. Đọc diff và policy trước, xác nhận chủ đích xóa; production cần review như một thay đổi phá hủy.

Câu 4. Image mới có cùng tag **latest**, Git không đổi. Argo có triển khai binary mới không? **Lời giải:** không có diff manifest để Argo buộc rollout; ngay cả khi Pod được thay, chính sách image pull/cache có thể tạo hành vi khác nhau. Dùng tag bất biến/digest và thay manifest qua Git để release truy vết được.

Câu 5. Người vận hành sửa nóng Deployment live, sau đó app tốt. Vì sao vẫn cần commit vào Git? **Lời giải:** source mong muốn vẫn là bản cũ; self-heal/sync sau đó có thể hoàn nguyên sửa nóng. Commit/review hoặc quy trình khẩn cấp hợp thức hóa trạng thái và tạo lịch sử cho người trực ca sau.

14. Bài tập tự đánh giá và tiêu chí đạt

- Vẽ luồng Git -> repo-server -> controller -> Kubernetes API -> kubelet/Pod -> request người dùng. Đánh dấu chỗ nào tạo manifest, chỗ nào áp manifest, chỗ nào thực sự chạy container.
- Giải thích bằng lời của bạn sự khác nhau giữa **OutOfSync**, **Degraded**, sync operation **Succeeded** và ứng dụng hoạt động đúng. Cho một ví dụ thực cho từng trường hợp.
- Thực hiện lab và lưu bốn bằng chứng: Application YAML, diff trước sync, **kubectl rollout status**, và một request thành công qua Service. Thông tin nhạy cảm phải được xóa khỏi ảnh/log.
- Tự gây hai lỗi: path không tồn tại và scale drift. Trước khi sửa, viết giả thuyết, bằng chứng xác nhận và tầng lỗi. Sau khi sửa, ghi điều gì thay đổi.
- Trả lời câu hỏi thiết kế: vì sao production cần branch protection, AppProject hạn chế quyền, image digest, smoke test và quy trình rollback dù Argo CD đã tự đồng bộ?

Bạn hiểu vững bài này khi có thể dự đoán trạng thái trước khi bấm Sync, tìm lỗi theo tầng bằng bằng chứng, và giải thích tác động của **prune**/**selfHeal** cho một đồng nghiệp. Nếu chỉ thuộc giao diện hoặc lệnh cài đặt, hãy làm lại phần drift và lỗi render.

15. Thuật ngữ và tài liệu chính thức

Desired state: trạng thái được Git/nguồn cấu hình định nghĩa. **Live state**: tài nguyên hiện có trong cluster. **Drift**: khác biệt giữa hai trạng thái. **Reconciliation**: vòng lặp so sánh và đưa live tới desired khi chính sách cho phép. **Sync**: áp desired manifests vào cluster. **Health**: đánh giá trạng thái hoạt động của resource theo quy tắc Argo. **Prune**: xóa resource không còn được quản lý trong source. **Self-heal**: tự sửa drift trong cluster. **AppProject**: ranh giới source/destination/resource và policy cho nhóm Application.

- Argo CD Getting Started
- Application specification
- Automated Sync Policy
- Projects
- Resource Health
- Official guestbook example

Phụ lục: mã lab để đọc offline

lesson/labs/argocd-local/guestbook-lab.yaml

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: guestbook-lab
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/argoproj/argocd-example-apps.git
    targetRevision: HEAD
    path: guestbook
  destination:
    server: https://kubernetes.default.svc
    namespace: guestbook-lab
  syncPolicy:
    syncOptions:
      - CreateNamespace=true
```